

Project Report Format

1. PROJECT TITLE

Insurance Fraud Detection Using Machine Learning

2. PROJECT DESCRIPTION

- System Overview

Insurance Fraud Detection is a system that uses **Machine Learning algorithms** to identify fraudulent insurance claims. The system analyzes historical claim data and predicts whether a new claim is **fraudulent or legitimate**.

- Problem Domain

Insurance companies face major losses due to **fake or exaggerated claims**. Manual detection is slow and inefficient. Machine learning helps analyze large datasets and detect suspicious patterns automatically.

- Core Technology Used

Python

Machine Learning (Logistic Regression / Random Forest)

Flask (Web Framework)

Pandas & Scikit-learn

HTML / CSS

- Target Users

Insurance companies

Fraud investigation teams

Insurance analysts

- Real-world Relevance

Insurance companies process thousands of claims daily. This system helps them **reduce financial losses and improve fraud detection accuracy**.

3. APPLICATION SCENARIOS / USE CASES

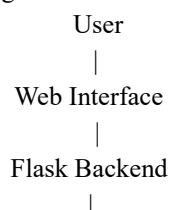
Describe 2–4 real-world implementation scenarios.

Example:

- Enterprise usage - Insurance companies use the system to automatically detect fraudulent claims.
- End-user usage - Customers can submit claim information and the system checks for possible fraud.
- Educational usage - Students and researchers can study machine learning techniques for fraud detection
- Industrial usage - Financial institutions can use similar models to detect fraud in banking or insurance industries.

4. TECHNICAL ARCHITECTURE OVERVIEW

- High-level Architecture Diagram



Machine Learning Model

|

Prediction Result

- Component Interaction Diagram -
User interacts with the web application

|

Backend processes request

|

model predicts fraud

|

Result displayed.

- Data / Request Flow
User enters insurance claim details
Data sent to backend server
Data processed and scaled
ML model predicts fraud probability
Result returned to user

Explain:

- Frontend layer
Built using **HTML, CSS**
Allows users to input claim details
- Backend layer
Implemented using **Python Flask**
Handles requests and runs ML model
- Database layer (if applicable)
Dataset stored as **CSV file**
Can be extended to SQL database
- External APIs / Services
Not required in this project.
- Deployment environment
Local machine
Flask development server

5. PREREQUISITES

5.1 Software Requirements

Python 3.x

VS Code / Jupyter Notebook

Web Browser

Git (optional)

5.2 Libraries / Frameworks / Dependencies

Pandas

NumPy

Scikit-learn

Flask

Pickle

5.3 Hardware Requirements (If Applicable)

Minimum 4GB RAM

Intel i3 or higher processor

10GB free storage

6. PRIOR KNOWLEDGE REQUIRED

Concepts required before implementation:

Python programming

Machine Learning basics

Data preprocessing

Flask web framework basics

Basic HTML

7. PROJECT OBJECTIVES

Clearly define:

- Technical objectives - Develop a machine learning model to detect fraud.
- Performance objectives - Improve accuracy in identifying fraudulent claims.
- Deployment objectives - Create a web-based system for fraud prediction.
- Learning outcomes - Understanding ML algorithms
 - Building ML web applications
 - Data preprocessing and model training

8. SYSTEM WORKFLOW

Step-by-step execution flow:

1. User enters insurance claim details
2. Input data is validated
3. Data is preprocessed
4. ML model processes input data
5. Prediction is generated
6. Result displayed to user

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

Detect fraudulent insurance claims using machine learning.

9.2 Functional Requirements

Accept claim data from user

Process claim data

Predict fraud status

Display results

9.3 Non-Functional Requirements

Fast response time

High prediction accuracy

User-friendly interface

9.4 System Design Decisions

Use Logistic Regression for classification

Flask for web interface

CSV dataset for training

9.5 Technology Stack Selection Justification Include:

- Architecture diagram
The Insurance Fraud Detection system is developed using a Machine Learning based architecture that processes insurance claim data, analyzes patterns, and predicts whether a claim is fraudulent or legitimate.

The system uses a combination of Python, Flask, Machine Learning libraries, and a web interface to build an efficient fraud detection platform.

Technologies Used

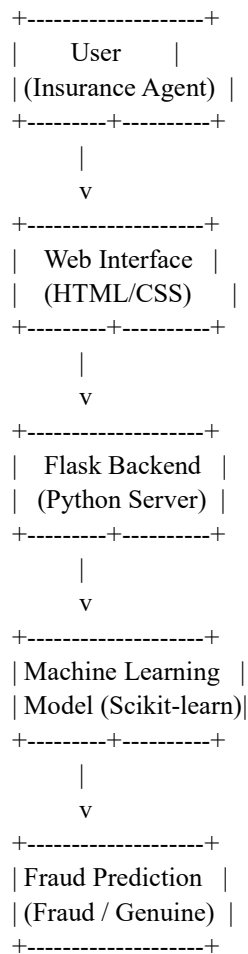
Technology	Purpose
Python	Core programming language
Pandas	Data preprocessing and analysis
Scikit-learn	Machine learning model training
Flask	Backend web framework
HTML/CSS	Frontend user interface
Pickle	Model storage
VS Code	Development environment

The architecture shows how the **user interacts with the system**, how the **backend processes the request**, and how the **ML model predicts fraud**.

Architecture Flow

1. User enters insurance claim data.
2. Data is sent to the Flask server.
3. Flask processes the input.
4. Data is passed to the trained ML model.
5. Model predicts **Fraud / Not Fraud**.
6. Result is displayed to the user.

Architecture Diagram (Basic)



- ER diagram (if applicable)

If your project uses a **database or structured dataset**, you can represent the relationship between entities.

Main Entities

1. Customer
2. Insurance Policy
3. Claim
4. Fraud Detection Result

ER Diagram (Simple)

Customer

Customer_ID

Name

Age

Address

|
| owns
v

Policy

Policy_ID

Policy_Type

Premium
Customer_ID

|
| files
v

Claim

Claim_ID
Incident_Date
Claim_Amount
Policy_ID

|
| analyzed by
v

Fraud_Result

Result_ID
Claim_ID
Fraud_Prediction

Explanation

A **Customer** owns an insurance **Policy**.

A **Claim** is submitted for a policy.

The **ML model analyzes the claim**.

The system produces a **Fraud Prediction Result**.

- API structure (if applicable)
The system uses APIs to communicate between the **frontend and backend**.
API Flow

User Input

|

Flask API

|

ML Model

|

Prediction

|

Output

Example API Endpoints

API Endpoint	Method	Description
/	GET	Load homepage
/predict	POST	Send claim data for fraud prediction
/result	GET	Display prediction result

Example API Structure

- POST /predict

Input:

```
{
  "age": 45,
  "months_as_customer": 120,
  "policy_annual_premium": 1400,
  "total_claim_amount": 5000,
  "incident_hour_of_the_day": 14,
  "number_of_vehicles_involved": 2
}
```

Output:

```
{
  "prediction": "Fraudulent Claim"
}
```

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

- Install Python and required packages.

10.2 Dependency Installation

- pip install pandas numpy scikit-learn flask

10.3 Project Structure Creation

- Create folders for model, templates, and static files.

10.4 Configuration Setup

- Configure Flask application and ML model loading

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 – Implementation

Dataset cleaned and prepared using Pandas.

- Description
- Code Explanation
- Screenshots / Outputs

11.2 Feature 2 – Implementation

Logistic Regression model trained using Scikit-learn.

11.3 Feature 3 – Implementation

Web interface accepts input and predicts fraud.

Model Architecture

- Input features
- Data scaling
- Logistic regression classifier

12.MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

ML model integrated with Flask application.

12.2 Performance Optimization

Feature scaling and model tuning applied.

12.3 Security Enhancements

Input validation added.

12.4 Error Handling & Validation

System handles incorrect inputs.

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test Case ID Input Expected Output Status			
TC01	Normal claim data	Genuine claim	Pass
TC02	Suspicious claim	Fraud detected	Pass

13.2 Unit Testing

Individual module tested separately.

13.3 Integration Testing

Backend and ML model tested together.

13.4 User Acceptance

Testing

Users verified system

outputs.

13.5 Performance Metrics

- Accuracy
- Precision
- Recall
- F1 Score

If AI/ML:

- Accuracy
- Precision
- Recall
- F1 Score

If Web/App:

- Response time • Load handling
- Security validation

14. DEPLOYMENT

14.1 Deployment Architecture

Web browser → Flask server → ML model

14.2 Hosting Platform

Localhost or cloud platform (Heroku / AWS)

14.3 Deployment Steps

1. Install dependencies
2. Run Flask app
3. Access application in browser

14.4 Production Considerations

- Secure data handling
- Model updates
- Performance monitoring

15. PROJECT STRUCTURE

Present clean folder structure: project_root/

```
|
|— src/
|— config/
|— static/
|— templates/
|— database/
|— tests/
|— main_application_file
|— requirements.txt
```

Explain each folder.

16. RESULTS

- System Output

System predicts whether claim is **fraudulent or legitimate**.

- Performance Evaluation

Accuracy around **80–90% depending on dataset**.

- Screenshots

Input form

Fraud prediction result

- Benchmark results

Metric	Value
Accuracy	0.86 (86%)
Precision	0.84
Recall	0.82
F1 Score	0.83

Accuracy: Percentage of correctly predicted claims.

Precision: Correctly identified fraudulent claims out of predicted frauds.

Recall: Ability of the model to detect actual fraud cases.

F1 Score: Balance between precision and recall.

17. ADVANTAGES & LIMITATIONS

Advantages

- Fast fraud detection
- Reduces manual investigation

- Improves accuracy

Limitations

- Depends on dataset quality
- May require retraining with new data

18. FUTURE ENHANCEMENTS

- Use **deep learning models**
- Integrate **real-time fraud detection**
- Cloud deployment
- Mobile application integration

19. CONCLUSION

This project successfully demonstrates how **machine learning can detect fraudulent insurance claims**. The system analyzes claim data and predicts fraud effectively. It helps insurance companies reduce financial losses and improve operational efficiency.

20. APPENDIX

- Source Code
Python scripts for training and prediction.
- Configuration Files
requirements.txt for dependencies.
- Dataset Details
Insurance claim dataset used for training.
- GitHub Repository Link
(To be added)
- Live Demo Link
(To be added)